Bài thực hành 2: BookStore với JSON Server (thay cho MockAPI)

> **Môn học:** Lập trình cho thiết bị di động – React Native **Mục tiêu:** Tự dựng một REST API giả lập ngay trên máy bằng **JSON Server**, có **đăng nhập thật bằng JWT** (json-server-auth), 3 resource `users`, `books`, `reviews`; sau đó cho app BookStore (Tuần 07) gọi API này. **Công cụ:** Node.js (LTS), `json-server@0.17.4`, `json-server-auth@2.1.0`, Expo, axios.

Mục lục

1. JSON Server là gì?
2. Dựng server từng bước
3. Thử API & các truy vấn hay dùng
4. Đăng nhập thật với json-server-auth
5. Kết nối từ app Expo
6. Đề bài thực hành 2
7. Hướng dẫn giải
8. Lỗi thường gặp

---

1. JSON Server là gì?

JSON Server là một gói npm biến **một file JSON** thành **một REST API đầy đủ** chỉ với một lệnh, không phải viết dòng code backend nào. Mỗi khoá cấp cao nhất trong file trở thành một resource:

{ "books": [...], "reviews": [...] }

sẽ có ngay các endpoint:

Phương thức	URL	Ý nghĩa
GET	`/books`	Lấy danh sách
GET	`/books/3`	Lấy 1 bản ghi

Phương thức	URL	Ý nghĩa
POST	`/books`	Thêm (id tự tăng)
PUT	`/books/3`	Thay toàn bộ bản ghi
PATCH	`/books/3`	Sửa một vài trường
DELETE	`/books/3`	Xoá

Mọi thay đổi (POST/PUT/PATCH/DELETE) được **ghi thẳng vào file `db.json`**, nên tắt server mở lại dữ liệu vẫn còn.

json-server-auth là phần mở rộng thêm 2 thứ mà MockAPI không có:

- Endpoint **`/register`** và **`/login`** thật: mật khẩu được băm bằng bcrypt, trả về **JWT** (`accessToken`) hết hạn sau **1 giờ**.
- **Phân quyền** theo route: ai được đọc, ai được ghi (cần token, chỉ chủ sở hữu...).

So sánh với MockAPI

	MockAPI (bản free)	JSON Server + json-server-auth
Chạy ở đâu	Trên Internet	**Trên máy tính của bạn** (cần bật server)
Số resource	Tối đa 2	**Không giới hạn**
Thiết kế review	Nhúng mảng `book.reviews`, gửi bằng PUT	**Resource riêng `/reviews`, gửi bằng POST** (đúng đề bài)
Đăng nhập	Phải giả lập ở client	**`/login`, `/register` thật, JWT, mật khẩu băm**
Phân quyền	Không có	Có (token, chủ sở hữu)
Lọc/sắp xếp	Lọc kiểu "chứa chuỗi"	**Lọc chính xác**, `_sort`, `_ne`, `_limit`, `_embed`...
Nhược điểm	Phụ thuộc mạng, giới hạn	Điện thoại phải **cùng Wi-Fi** với máy chạy server

> ⚠️ **Phiên bản:** lệnh `npm i json-server` hiện cài bản **1.0.0-beta**, bản này đã đổi cú pháp truy vấn và **không chạy được với json-server-auth**. Bài này cố định `json-server@0.17.4` (bản ổn định, tài liệu nhiều nhất).

---

2. Dựng server từng bước

Bước 2.1: Kiểm tra Node.js

node -v   # cần v18 trở lên

npm -v

Chưa có thì tải bản LTS tại https://nodejs.org.

Bước 2.2: Tạo project server

Tạo thư mục **riêng**, nằm ngoài project Expo:

mkdir bookstore-api

cd bookstore-api

npm init -y

npm install json-server@0.17.4 json-server-auth@2.1.0

Cấu trúc thư mục khi hoàn thành:

bookstore-api/

├── db.json   # dữ liệu "sống" (server đọc + ghi vào đây)

├── db.seed.json   # bản sao gốc để khôi phục

├── routes.json   # phân quyền

└── package.json

Bước 2.3: Tạo dữ liệu `db.json`

Thiết kế 3 resource và quan hệ giữa chúng:

users (id, name, email, password)

  │ 1

  │   n

  └──────────────── reviews (id, bookId, userId, userName, rating, comment, createdAt)

                │ n

books (id, ...) ──────┘ 1

- `reviews.bookId` → `books.id` và `reviews.userId` → `users.id` là **khoá ngoại**. Tên phải đúng dạng `<tên số ít>Id` thì `_embed`/`_expand` mới hoạt động.
- `id` để kiểu **number**: json-server tự tăng `id` khi POST.
- `userName` được lưu **thừa** (denormalize) trong review để hiển thị tên mà không phải đọc bảng `users` (bảng này được bảo vệ, xem Bước 2.4).
- **Mật khẩu phải là chuỗi băm bcrypt**, không được ghi `"123456"` trực tiếp, vì json-server-auth so sánh bằng bcrypt. Chuỗi `$2a$10$pDEa...` trong file mẫu là mật khẩu `123456`.

Trích một phần `db.json` (file đầy đủ có 4 users, 20 books thuộc 4 thể loại, 25 reviews):


```
{
  "users": [
    {
      "id": 1,
      "name": "Nguyễn Văn An",
      "email": "an@gmail.com",
      "password":
"$2a$10$pDEa13HpLHUEnHXVKrzu4.wsRemAMTMCSW2j96iRhx2EhjKk/v0P2"
    }
  ],
```

```
  "books": [
    {
      "id": 1,
      "title": "Dế Mèn Phiêu Lưu Ký",
      "author": "Tô Hoài",
      "category": "Văn học",
      "price": 45000,
      "image": "https://picsum.photos/seed/book1/300/450",
      "description": "Dế Mèn Phiêu Lưu Ký của Tô Hoài, một cuốn sách thuộc thể loại văn học được nhiều bạn đọc yêu thích."
    }
  ],
  "reviews": [
    {
      "id": 1,
      "bookId": 1,
      "userId": 1,
      "userName": "Nguyễn Văn An",
      "rating": 2,
      "comment": "Hơi khó đọc với mình.",
      "createdAt": "2026-10-03T09:00:00.000Z"
    },
```

```
    {
      "id": 2,
      "bookId": 1,
      "userId": 2,
      "userName": "Trần Thị Bình",
      "rating": 5,
      "comment": "Tuyệt vời, mình đọc 2 lần rồi.",
      "createdAt": "2026-10-02T09:00:00.000Z"
    }
  ]
}
```

Tài khoản có sẵn (mật khẩu đều là `123456`): `an@gmail.com`, `binh@gmail.com`, `chau@gmail.com`, `test@gmail.com`.

> Muốn tự thêm user với mật khẩu khác thì tạo chuỗi băm bằng lệnh (chạy trong thư mục `bookstore-api`):
>
> node -e "console.log(require('bcryptjs').hashSync('matkhaumoi', 10))"
>
> Cách dễ hơn là gọi `POST /register` (mục 4): server tự băm và tự ghi vào `db.json`.

Sau khi có `db.json`, tạo bản sao để khôi phục khi cần:

cp db.json db.seed.json # Windows: copy db.json db.seed.json

Bước 2.4: Phân quyền `routes.json`

```
{
  "users": 600,
```

```
  "books": 444,
  "reviews": 644
}
```

Mỗi con số gồm 3 chữ số, giống quyền file trên Linux: **chủ sở hữu – người đã đăng nhập – khách**. Mỗi chữ số: `6` = đọc + ghi, `4` = chỉ đọc, `0` = không có quyền.

Route	Mã	Chủ sở hữu	Đã đăng nhập	Khách	Áp dụng
`users`	600	đọc/ghi	✗	✗	Chỉ xem được **chính mình**, cần token
`books`	444	đọc	đọc	đọc	Ai cũng xem, **không ai sửa** được qua API
`reviews`	644	đọc/ghi	đọc	đọc	Ai cũng xem; **tạo/sửa/xoá phải có token** và `userId` = chủ token

> "Chủ sở hữu" của một bản ghi được xác định bằng trường **`userId`** trong bản ghi đó, so với `id` của user trong token.

Bước 2.5: Thêm script vào `package.json`

```
{
  "name": "bookstore-api",
  "version": "1.0.0",
  "description": "json-server + json-server-auth cho bài thực hành BookStore",
  "scripts": {
```

```
    "start": "json-server-auth db.json -r routes.json --port 3000 --host 0.0.0.0",

    "reset": "node -e \"require('fs').copyFileSync('db.seed.json','db.json'); console.log('Đã khôi phục db.json')\""

  },

  "dependencies": {

    "json-server": "0.17.4",

    "json-server-auth": "2.1.0"

  },

  "private": true

}
```

- `--host 0.0.0.0`: cho phép máy khác trong mạng LAN (điện thoại) gọi vào. Mặc định server chỉ nhận từ `localhost`.
- `npm run reset`: chép `db.seed.json` đè lên `db.json`. **Phải tắt server trước khi reset**, vì server giữ dữ liệu trong bộ nhớ và sẽ ghi đè lại file.

Bước 2.6: Chạy server

npm start

Kết quả mong đợi:

```
  \{^_^}/ hi!

  Loading db.json

  Loading /tmp/routes-from-bookstore-api.json

  Loading .../node_modules/json-server-auth/dist

  Done

  Resources
```

http://localhost:3000/users

http://localhost:3000/books

http://localhost:3000/reviews

Other routes

/users* -> /600/users$1

/books* -> /444/books$1

/reviews* -> /644/reviews$1

Mở trình duyệt vào http://localhost:3000/books để thấy danh sách sách. Bấm `Ctrl + C` trong terminal để tắt server.

---

3. Thử API & các truy vấn hay dùng

Có thể thử bằng trình duyệt (chỉ GET), extension **Thunder Client** / **REST Client** trong VS Code, Postman, hoặc `curl`.

3.1. Truy vấn GET (đã chạy thử với dữ liệu mẫu)

Mục đích	**Request**	**Kết quả**
Một cuốn sách	`GET /books/3`	Tắt Đèn
Lọc chính xác	`GET /books?category=Kỹ năng`	5 cuốn
Sách liên quan (trừ sách đang xem)	`GET /books?category=Kỹ năng&id_ne=6`	4 cuốn
Review của 1 sách, mới nhất trước	`GET /reviews?bookId=11&_sort=createdAt&_order=desc`	3 review
Review của sách 1	`GET /reviews?bookId=1`	2 review, **chỉ của sách 1** (không lẫn 10, 11 như MockAPI)

Mục đích	Request	Kết quả
Route lồng nhau	`GET /books/1/reviews`	giống dòng trên
Nhúng con vào cha	`GET /books/1?_embed=reviews`	sách 1 kèm mảng `reviews`
Phân trang	`GET /books?_page=2&_limit=5`	sách 6–10 (header `X-Total-Count` = 20)
Tìm kiếm toàn văn	`GET /books?q=thói quen`	mọi trường có chứa "thói quen"
Khoảng giá	`GET /books?price_gte=100000&price_lte=200000`	giá từ 100k đến 200k
Chứa chuỗi (regex)	`GET /books?title_like=^Đ`	tên bắt đầu bằng "Đ"

Hậu tố toán tử: `_ne` (khác), `_gte` (≥), `_lte` (≤), `_like` (regex). Tham số đặc biệt: `_sort`, `_order` (`asc`/`desc`), `_page`, `_limit`, `_start`/`_end`, `_embed`, `_expand`, `q`.

> ⚠️ **Không dùng `GET /reviews?_expand=user`**: lệnh này nhúng cả bản ghi user vào kết quả, **kể cả chuỗi băm mật khẩu**. Đó là lý do review lưu sẵn `userName`.

3.2. Ghi dữ liệu với `curl`

```
# Khách tạo review → bị chặn

curl -X POST http://localhost:3000/reviews \

  -H "Content-Type: application/json" \

  -d '{"bookId":8,"userId":4,"rating":5}'

# → 401 "Missing authorization header"

# Sửa sách → bị chặn (444 = chỉ đọc)

curl -X POST http://localhost:3000/books -H "Content-Type: application/json" -d '{"title":"x"}'
```

→ 403 "Read only"

---

4. Đăng nhập thật với json-server-auth

4.1. Đăng ký `POST /register`

curl -X POST http://localhost:3000/register \

-H "Content-Type: application/json" \

-d '{"email":"moi@gmail.com","password":"123456","name":"Sinh viên Mới"}'

{

"accessToken":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Im1vaUBnbWFpbC5jb20i...",

"user": { "email": "moi@gmail.com", "name": "Sinh viên Mới", "id": 5 }

}

Bắt buộc có `email` và `password`. Các trường khác (`name`...) được lưu nguyên vào `users`.

4.2. Đăng nhập `POST /login`

curl -X POST http://localhost:3000/login \

-H "Content-Type: application/json" \

-d '{"email":"test@gmail.com","password":"123456"}'

{

"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",

"user": { "id": 4, "name": "Tài khoản Test", "email": "test@gmail.com" }

}

4.3. Gọi API cần quyền: gửi kèm token

```
TOKEN="eyJhbGciOi..."   # accessToken vừa nhận

# Xem chính mình → 200

curl http://localhost:3000/users/4 -H "Authorization: Bearer $TOKEN"

# Tạo review của chính mình → 201, server trả về review có id mới

curl -X POST http://localhost:3000/reviews \

  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \

  -d '{"bookId":8,"userId":4,"userName":"Tài khoản
Test","rating":5,"comment":"Hay","createdAt":"2026-10-19T07:00:00.000Z"}'
```

4.4. Bảng thông báo lỗi (đã chạy thử)

Tình huống	**Mã**	**Nội dung server trả về**
Email chưa đăng ký	400	`Cannot find user`
Sai mật khẩu	400	`Incorrect password`
Đăng ký email đã có	400	`Email already exists`
Mật khẩu < 4 ký tự	400	`Password is too short`
Không gửi token	401	`Missing authorization header`
Token sai	401	`jwt malformed`
Token hết hạn (sau 1 giờ)	401	`jwt expired`
Xem user khác	403	`Private resource access: entity must have a reference to the owner id`
Tạo review với `userId` của người khác	403	`Private resource creation: request body`

Tình huống	Mã	Nội dung server trả về
		`must have a reference to the owner id`
Ghi vào `books`	403	`Read only`

> Server trả lỗi dạng **chuỗi** (không phải JSON object), nên trong app đọc bằng `err.response.data`.

4.5. JWT là gì?

`accessToken` có 3 phần ngăn bởi dấu chấm: `header.payload.chữ_ký`. Phần **payload** là JSON mã hoá base64url, chỉ được mã hoá chứ **không** bị giấu đi:

{ "email": "test@gmail.com", "iat": 1792394979, "exp": 1792398579, "sub": "4" }

- `sub`: id của user, server dùng nó để biết "chủ sở hữu".
- `exp`: thời điểm hết hạn, tính bằng **giây** (ở đây sau `iat` đúng 3600 giây = 1 giờ). App đọc `exp` để biết token còn hạn hay không khi auto-login.
- Chữ ký do server tạo bằng khoá bí mật. Sửa payload thì chữ ký sai, server trả 401.

---

5. Kết nối từ app Expo

5.1. Địa chỉ server theo từng cách chạy app

App chạy trên	Địa chỉ dùng trong app
Điện thoại thật (Expo Go)	`http://<IP-LAN-của-máy-tính>:3000`
Android Emulator	`http://10.0.2.2:3000` (10.0.2.2 = máy tính chủ)
iOS Simulator	`http://localhost:3000`
Web: trình duyệt, MobileView trong VS Code	`http://localhost:3000`

> **Trên điện thoại, `localhost` là chính cái điện thoại**, không phải máy tính. Đây là lỗi hay gặp nhất.

5.2. Lấy IP LAN của máy tính

macOS

ipconfig getifaddr en0

Windows (xem dòng IPv4 Address của card Wi-Fi)

ipconfig

Expo cũng in sẵn IP này khi chạy `npx expo start`, ở dòng `exp://192.168.x.x:8081`.

5.3. Kiểm tra trước khi code

1. Máy tính và điện thoại **cùng mạng Wi-Fi**. Wi-Fi trường thường chặn các máy nói chuyện với nhau; khi đó dùng điểm phát hotspot từ điện thoại.
2. Mở **trình duyệt trên điện thoại**, vào `http://<IP>:3000/books`. Thấy JSON nghĩa là đã thông.
3. Không thấy gì:
    - **macOS:** khi hệ thống hỏi cho phép "node" nhận kết nối đến thì bấm Allow.
    - **Windows:** cho phép Node.js qua Windows Defender Firewall (Private network).

5.4. Cấu hình trong app `src/api/client.ts`

// src/api/client.ts

import axios from 'axios';

import { Platform } from 'react-native';

// ===== Địa chỉ json-server =====

// - Điện thoại thật (Expo Go): IP LAN của máy tính chạy json-server, cùng Wi-Fi

//   (macOS: ipconfig getifaddr en0 | Windows: ipconfig → IPv4 Address)

// - Android Emulator: 10.0.2.2 trỏ về máy tính

// - iOS Simulator / chạy web (MobileView, trình duyệt): localhost

const LAN_IP = '192.168.1.10'; // TODO: đổi thành IP máy của bạn

const PORT = 3000;

```
const USE_REAL_DEVICE = true; // false khi dùng emulator/simulator/web

const host = USE_REAL_DEVICE

  ? LAN_IP

  : Platform.OS === 'android'

    ? '10.0.2.2'

    : 'localhost';

export const BASE_URL = `http://${host}:${PORT}`;

export const api = axios.create({

  baseURL: BASE_URL,

  timeout: 10000,

});

// Gắn token vào mọi request. json-server-auth đọc header này để phân quyền

let authToken: string | null = null;

export const setAuthToken = (token: string | null) => {

  authToken = token;

};

api.interceptors.request.use((config) => {

  if (authToken) {

    config.headers.Authorization = `Bearer ${authToken}`;

  }

  return config;
```

```
});

// Lấy thông báo lỗi từ json-server-auth (server trả về chuỗi, vd "Incorrect password")

export const getServerMessage = (err: any): string | undefined =>

  typeof err?.response?.data === 'string' ? err.response.data : undefined;
```

> Expo Go cho phép gọi `http://` (không mã hoá) trên cả Android và iOS. CORS đã được json-server bật sẵn, nên bản web cũng gọi được.

---

6. Đề bài thực hành 2

Làm lại các yêu cầu Tuần 07 trên app BookStore, nhưng dùng **API JSON Server tự dựng** (thư mục `bookstore-api`) thay cho MockAPI.

Bài tập 1: Đăng nhập / Đăng ký với JWT thật

- Form giữ nguyên yêu cầu validate: email đúng định dạng, mật khẩu ≥ 6 ký tự, lỗi hiện dưới từng ô.
- `authService.login` gọi `POST /login`, `authService.register` gọi `POST /register`.
- Đổi thông báo lỗi tiếng Anh của server ("Incorrect password", "Email already exists"...) sang tiếng Việt.
- Lưu `accessToken` + `user` vào store `auth` và AsyncStorage. **Không lưu mật khẩu.**

Bài tập 2: Auto-login có xác minh token

- Đọc thời điểm hết hạn `exp` trong JWT; token hết hạn thì không auto-login.
- Còn hạn thì gọi `GET /users/:id` kèm token. Nhận 401 hoặc 403 thì xoá phiên, coi như chưa đăng nhập.
- Đăng xuất và bảo vệ Checkout giống Tuần 07.

Bài tập 3: Review là resource riêng

- `GET /reviews?bookId=..&_sort=createdAt&_order=desc` để hiển thị danh sách + điểm trung bình (dùng lại `RatingStars`).
- Gửi review bằng **`POST /reviews`** kèm token. Thành công thì chèn review vào đầu danh sách, không load lại trang.
- Token hết hạn khi đang gửi (401) thì tự đăng xuất và hiện nút "Đăng nhập để viết đánh giá".

Thử thách

- "Sách liên quan" lọc **trên server**: `GET /books?category=..&id_ne=..&_limit=10`.
- (Nâng cao) Cho phép người dùng **xoá review của chính mình** (`DELETE /reviews/:id`; server tự chặn review của người khác bằng 403). Nút xoá chỉ hiện khi `review.userId === user.id`.
- (Nâng cao) Phân trang danh sách sách ở Home bằng `_page` + `_limit`, kéo xuống cuối thì tải thêm (`onEndReached`).

Kiểm tra bắt buộc: đăng nhập bằng `test@gmail.com` / `123456`, gửi 1 review cho sách "Tư Duy Nhanh Và Chậm" (id 8), rồi mở http://localhost:3000/reviews?bookId=8 để thấy bản ghi mới trong `db.json`.

---

7. Hướng dẫn giải

Lời giải dựa trên project Tuần 07 (thư mục `BookStoreApp_JsonServer`). Màn hình, store, navigation, `RatingStars`, `ReviewForm`... **giữ nguyên**. Chỉ phần nói chuyện với API thay đổi. Đây là lợi ích của việc tách **service layer**.

File	Thay đổi so với bản MockAPI
`src/types.ts`	`id` kiểu `number`; `Book` bỏ `reviews`; `Review` có `bookId`
`src/api/client.ts`	Địa chỉ LAN/emulator; hàm `getServerMessage`
`src/services/authService.ts`	Gọi `/login`, `/register` thật; đọc `exp` từ JWT; dịch lỗi
`src/services/bookService.ts`	`getRelated` lọc trên server bằng `id_ne`, `_limit`
`src/services/reviewService.ts`	`getByBook` + `create` bằng **POST**
`src/store/authSlice.ts`	Auto-login: 401/403/404 thì xoá phiên
`src/screens/BookDetailScreen.tsx`	Tải sách + review song song; POST xong chèn vào đầu; 401 thì đăng xuất

File	Thay đổi so với bản MockAPI
`keyExtractor`, `cartSlice`	`String(item.id)` vì `id` giờ là number

Bước 7.1: Kiểu dữ liệu `src/types.ts`

```
// src/types.ts

// json-server dùng id kiểu number (tự tăng 1, 2, 3...)

export type Id = number;

export type User = {

  id: Id;

  name: string;

  email: string;

};

export type Book = {

  id: Id;

  title: string;

  author: string;

  category: string;

  price: number;

  image: string;

  description: string;

};

// Resource riêng /reviews, liên kết với sách qua bookId (khoá ngoại)

export type Review = {
```

```
  id: Id;
  bookId: Id;
  userId: Id;
  userName: string;
  rating: number; // 1..5
  comment: string;
  createdAt: string; // ISO string
};
```

> Đổi `id` sang `number` thì TypeScript báo lỗi ngay ở những chỗ cần sửa, ví dụ `keyExtractor` bắt buộc trả về **string** nên phải viết `keyExtractor={(item) => String(item.id)}`. Đây là lợi ích của TypeScript: cứ sửa theo danh sách lỗi `npx tsc --noEmit` là xong.

Bước 7.2: Đăng nhập / Đăng ký `src/services/authService.ts`

```
// src/services/authService.ts
import { api, getServerMessage } from '../api/client';
import { User } from '../types';
export type AuthResult = {
  token: string;
  user: User;
  expiresAt: number; // mốc thời gian (ms) token hết hạn, đọc từ JWT
};
type AuthResponse = { accessToken: string; user: User & { password?: string } };
// JWT gồm 3 phần header.payload.signature; payload là JSON mã hoá base64url,
```

```
// trong đó "exp" là thời điểm hết hạn (tính bằng GIÂY)
function getJwtExpiry(token: string): number {
  try {
    const base64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
    const payload = JSON.parse(atob(base64));
    return payload.exp * 1000;
  } catch {
    return Date.now() + 60 * 60 * 1000; // không đọc được → coi như 1 giờ
  }
}
// Không bao giờ giữ mật khẩu (dù đã băm) trong app
const toPublicUser = ({ password, ...user }: User & { password?: string }): User => user;
const toResult = (data: AuthResponse): AuthResult => ({
  token: data.accessToken,
  user: toPublicUser(data.user),
  expiresAt: getJwtExpiry(data.accessToken),
});
// Đổi thông báo tiếng Anh của json-server-auth sang tiếng Việt
const MESSAGES: Record<string, string> = {
  'Cannot find user': 'Email hoặc mật khẩu không đúng',
  'Incorrect password': 'Email hoặc mật khẩu không đúng',
```

```
  'Email already exists': 'Email đã được sử dụng',
  'Email format is invalid': 'Email không đúng định dạng',
  'Password is too short': 'Mật khẩu quá ngắn',
};
const toAppError = (err: unknown) => {
  const msg = getServerMessage(err);
  if (msg) return new Error(MESSAGES[msg] ?? msg);
  return new Error('Không kết nối được máy chủ. Kiểm tra json-server và địa chỉ IP');
};
async function login(email: string, password: string): Promise<AuthResult> {
  try {
    const res = await api.post<AuthResponse>('/login', { email: email.trim(), password });
    return toResult(res.data);
  } catch (err) {
    throw toAppError(err);
  }
}
async function register(name: string, email: string, password: string): Promise<AuthResult> {
  try {
    const res = await api.post<AuthResponse>('/register', {
      name: name.trim(),
      email: email.trim().toLowerCase(),
```

```
      password,
    });
    return toResult(res.data);
  } catch (err) {
    throw toAppError(err);
  }
}
// Xác minh token khi auto-login: route /users là 600 → chỉ chủ tài khoản đọc được.
// Token sai/hết hạn → 401, token của người khác → 403
async function getMe(userId: number): Promise<User> {
  const res = await api.get<User & { password?: string }>(`/users/${userId}`);
  return toPublicUser(res.data);
}
export default { login, register, getMe };
```

Bước 7.3: Auto-login xác minh token `src/store/authSlice.ts` (đoạn `restoreSession`)

```
// src/store/authSlice.ts (trích)
// Auto-login: đọc AsyncStorage khi mở app
export const restoreSession = createAsyncThunk<AuthResult | null>('auth/restore', async () => {
  try {
    const raw = await AsyncStorage.getItem(AUTH_STORAGE_KEY);
    if (!raw) return null;
```

```
  const saved: AuthResult = JSON.parse(raw);
  const isValid = !!saved.token && !!saved.user?.id && saved.expiresAt > Date.now();
  if (!isValid) {
    await AsyncStorage.removeItem(AUTH_STORAGE_KEY); // token hết hạn → xoá
    return null;
  }
  setAuthToken(saved.token);
  // Xác minh token với server: GET /users/:id (route 600, cần token hợp lệ)
  try {
    const user = await authService.getMe(saved.user.id);
    return { ...saved, user };
  } catch (err: any) {
    // 401: token sai/hết hạn · 403: không phải chủ tài khoản · 404: user đã bị xoá
    if ([401, 403, 404].includes(err?.response?.status)) {
      await AsyncStorage.removeItem(AUTH_STORAGE_KEY);
      setAuthToken(null);
      return null;
    }
    return saved; // mất mạng: vẫn cho vào bằng dữ liệu đã lưu
  }
} catch {
```

```
    return null; // dữ liệu hỏng (JSON lỗi) → coi như chưa đăng nhập
  }
});
```

Bước 7.4: Sách `src/services/bookService.ts`

```
// src/services/bookService.ts

import { api } from '../api/client';

import { Book } from '../types';

async function getAll(): Promise<Book[]> {
  const res = await api.get<Book[]>('/books');
  return res.data;
}

async function getById(id: number): Promise<Book> {
  const res = await api.get<Book>(`/books/${id}`);
  return res.data;
}

// Sách liên quan: lọc NGAY TRÊN SERVER
//   category=...  : cùng danh mục (so khớp chính xác)
//   id_ne=...     : trừ sách hiện tại (_ne = not equal)
//   _limit=10     : tối đa 10 cuốn
async function getRelated(book: Book, limit = 10): Promise<Book[]> {
  const res = await api.get<Book[]>('/books', {
    params: { category: book.category, id_ne: book.id, _limit: limit },
```

```
  });
  return res.data;
}
export default { getAll, getById, getRelated };
```

Bước 7.5: Review `src/services/reviewService.ts`

```
// src/services/reviewService.ts
import { api } from '../api/client';
import { Review } from '../types';
export type NewReview = Omit<Review, 'id' | 'createdAt'>;
// GET /reviews?bookId=3&_sort=createdAt&_order=desc
// (tương đương GET /books/3/reviews nhưng có sắp xếp mới nhất lên đầu)
async function getByBook(bookId: number): Promise<Review[]> {
  const res = await api.get<Review[]>('/reviews', {
    params: { bookId, _sort: 'createdAt', _order: 'desc' },
  });
  return res.data;
}
// POST /reviews — route 644: ai cũng đọc được, nhưng muốn tạo phải
// gửi kèm token VÀ userId trong body phải trùng với chủ token
async function create(input: NewReview): Promise<Review> {
  const res = await api.post<Review>('/reviews', {
```

```
    ...input,
    createdAt: new Date().toISOString(),
  });
  return res.data; // json-server trả về bản ghi vừa tạo (có id mới)
}
export default { getByBook, create };
```

Bước 7.6: Màn chi tiết `src/screens/BookDetailScreen.tsx`

```
// src/screens/BookDetailScreen.tsx
import React, { useCallback, useEffect, useMemo, useState } from 'react';
import { ActivityIndicator, Image, ScrollView, StyleSheet, Text, View } from 'react-native';
import RatingStars from '../components/RatingStars';
import ReviewItem from '../components/ReviewItem';
import ReviewForm from '../components/ReviewForm';
import RelatedBooks from '../components/RelatedBooks';
import PrimaryButton from '../components/PrimaryButton';
import bookService from '../services/bookService';
import reviewService from '../services/reviewService';
import { ScreenProps } from '../navigation/types';
import { useAppDispatch, useAppSelector } from '../store';
import { addToCart } from '../store/cartSlice';
import { logout } from '../store/authSlice';
import { Book, Review } from '../types';
```

```
import { averageRating } from '../utils/rating';

export default function BookDetailScreen({ navigation, route }: ScreenProps<'BookDetail'>) {

  const { bookId } = route.params;

  const dispatch = useAppDispatch();

  const user = useAppSelector((s) => s.auth.user);

  const [book, setBook] = useState<Book | null>(null);

  const [reviews, setReviews] = useState<Review[]>([]);

  const [related, setRelated] = useState<Book[]>([]);

  const [loading, setLoading] = useState(true);

  const [error, setError] = useState<string | null>(null);

  useEffect(() => {

    let cancelled = false; // tránh setState khi đã rời màn hình

    (async () => {

      try {

        setLoading(true);

        // Gọi song song: thông tin sách + danh sách review của sách

        const [b, r] = await Promise.all([bookService.getById(bookId),
reviewService.getByBook(bookId)]);

        if (cancelled) return;

        setBook(b);

        setReviews(r);

        // Thử thách: sách liên quan (lỗi phần này không làm hỏng cả trang)
```

```
      bookService.getRelated(b).then((list) => !cancelled && setRelated(list)).catch(() => {});
    } catch {
      if (!cancelled) setError('Không tải được thông tin sách');
    } finally {
      if (!cancelled) setLoading(false);
    }
  })();
  return () => {
    cancelled = true;
  };
}, [bookId]);
// Điểm trung bình tự tính lại mỗi khi mảng reviews thay đổi
const avg = useMemo(() => averageRating(reviews), [reviews]);
const handleSubmitReview = useCallback(
  async (rating: number, comment: string) => {
    if (!user) return;
    let created: Review;
    try {
      // POST /reviews → server trả về review vừa tạo
      created = await reviewService.create({
        bookId,
```

```
          userId: user.id, // phải trùng chủ token, nếu không server trả 403
          userName: user.name,
          rating,
          comment,
        });
      } catch (err: any) {
        // Token hết hạn (json-server-auth: 1 giờ) → đăng xuất, form đổi thành nút "Đăng nhập"
        if (err?.response?.status === 401) dispatch(logout());
        throw err; // để ReviewForm hiện thông báo lỗi
      }
      // Thêm vào state cục bộ → danh sách + điểm TB đổi ngay, không load lại cả trang
      setReviews((prev) => [created, ...prev]);
    },
    [bookId, user, dispatch],
  );
  if (loading) return <ActivityIndicator style={{ flex: 1 }} size="large" />;
  if (error || !book) return <Text style={styles.center}>{error ?? 'Không tìm thấy sách'}</Text>;
  return (
    <ScrollView contentContainerStyle={styles.container}
keyboardShouldPersistTaps="handled">
      <View style={styles.top}>
        <Image source={{ uri: book.image }} style={styles.cover} />
```

```
  <View style={styles.info}>
    <Text style={styles.title}>{book.title}</Text>
    <Text style={styles.author}>{book.author}</Text>
    <Text style={styles.category}>{book.category}</Text>
    <Text style={styles.price}>{book.price.toLocaleString('vi-VN')} đ</Text>
    <PrimaryButton title="Thêm vào giỏ" onPress={() => dispatch(addToCart(book))} />
  </View>
</View>
<Text style={styles.desc}>{book.description}</Text>
{/* ---- Bài 3: Điểm trung bình + danh sách review ---- */}
<Text style={styles.sectionTitle}>Đánh giá</Text>
<View style={styles.avgRow}>
  <Text style={styles.avgNumber}>{avg.toFixed(1)}</Text>
  <View>
    <RatingStars value={avg} size={22} />
    <Text style={styles.count}>{reviews.length} đánh giá</Text>
  </View>
</View>
{user ? (
  <ReviewForm onSubmit={handleSubmitReview} />
) : (
  <PrimaryButton
```

```
          variant="outline"
          title="Đăng nhập để viết đánh giá"
          onPress={() => navigation.navigate('Login', { redirect: 'goBack' })}
        />
      )}
      {reviews.length === 0 ? (
        <Text style={styles.empty}>Chưa có đánh giá nào. Hãy là người đầu tiên!</Text>
      ) : (
        // Trong ScrollView dùng map (không lồng FlatList dọc trong ScrollView dọc)
        reviews.map((r) => <ReviewItem key={r.id} review={r} />)
      )}
      {/* ---- Thử thách: Sách liên quan ---- */}
      <RelatedBooks books={related} onPressBook={(id) => navigation.push('BookDetail', { bookId: id })} />
    </ScrollView>
  );
}
const styles = StyleSheet.create({
  container: { padding: 16, paddingBottom: 40 },
  center: { flex: 1, textAlign: 'center', marginTop: 40 },
  top: { flexDirection: 'row', gap: 14 },
  cover: { width: 120, height: 180, borderRadius: 8, backgroundColor: '#eee' },
```

```
  info: { flex: 1, gap: 4 },
  title: { fontSize: 20, fontWeight: '700' },
  author: { color: '#555' },
  category: { color: '#1565c0' },
  price: { fontSize: 18, color: '#c62828', fontWeight: '700', marginBottom: 8 },
  desc: { marginTop: 16, lineHeight: 21, color: '#333' },
  sectionTitle: { fontSize: 18, fontWeight: '700', marginTop: 24, marginBottom: 8 },
  avgRow: { flexDirection: 'row', alignItems: 'center', gap: 12, marginBottom: 12 },
  avgNumber: { fontSize: 40, fontWeight: '800' },
  count: { color: '#666', marginTop: 2 },
  empty: { color: '#888', marginTop: 12, fontStyle: 'italic' },
});
```

Bước 7.7: Chạy thử

```
# Terminal 1
cd bookstore-api
npm start
# Terminal 2
cd BookStoreApp_JsonServer
npm install
npx expo install --fix
npx expo start
```

Nhớ sửa `LAN_IP` / `USE_REAL_DEVICE` trong `src/api/client.ts` trước khi chạy.

---

8. Lỗi thường gặp

Hiện tượng	**Nguyên nhân**	**Cách sửa**
App báo `Network Error`	Server chưa chạy; sai IP; dùng `localhost` trên điện thoại; khác Wi-Fi; bị tường lửa chặn	Kiểm tra theo mục 5.3, thử mở `http://IP:3000/books` trên trình duyệt điện thoại
`json-server-auth` báo lỗi khi khởi động	Đã cài `json-server` bản 1.0 beta	`npm i json-server@0.17.4`
Đăng nhập luôn báo `Incorrect password` với user tự thêm	Ghi mật khẩu dạng chữ thường vào `db.json`	Dùng `/register` hoặc tạo chuỗi băm bcrypt (mục 2.3)
`401 Missing authorization header`	Quên gắn token	Kiểm tra `setAuthToken` sau login / restore
`403 Private resource creation...`	`userId` trong body khác user của token	Gửi `userId: user.id` của người đang đăng nhập
Dùng app được một lúc thì gửi review báo lỗi	Token hết hạn sau 1 giờ (`jwt expired`)	Bắt 401 → `logout()` → đăng nhập lại
`npm run reset` xong vẫn thấy dữ liệu cũ	Server đang chạy giữ dữ liệu trong bộ nhớ	Tắt server → reset → bật lại
Lỗi TypeScript ở `keyExtractor`	`id` là number	`String(item.id)`
`GET /reviews?bookId=1` trả về rỗng	Lưu `bookId` dạng chuỗi `"1"` ở một số bản ghi	Thống nhất kiểu number trong `db.json`
Thấy chuỗi băm mật khẩu trong kết quả	Dùng `_expand=user` hoặc giữ nguyên object từ `GET /users/:id`	Không dùng `_expand=user`; lọc bỏ `password` (`toPublicUser`)